SRE event handling

Introduction to Site Reliability Engineering and the Incident Response Landscape

The discipline of Site Reliability Engineering (SRE) emerged from a fundamental need to bridge the historical divide between software development and IT operations. By treating operational problems as software engineering problems, SRE teams focus on automating toil, managing system dependencies, and maintaining reliability through carefully negotiated Service Level Objectives (SLOs) and error budgets[1]. However, despite rigorous preparation, production systems exist in an imperfect world where outages are inevitable. When a critical service fails, the daily routine of capacity planning and infrastructure coding is immediately replaced by the high-stakes, time-compressed reality of incident management.

Real SRE work during a production outage is characterized by immense cognitive load, fragmented information landscapes, and extreme pressure to restore service. The basic premise of incident management is to respond to these chaotic events in a structured, predefined manner. Large-scale incidents can quickly become confusing, requiring a clear line of command and designated roles to maintain order[2]. Industry best practices dictate the use of frameworks built around the "3Cs" of incident management: coordinating the response effort, communicating between responders and the outside world, and maintaining control over the incident[2]. Roles are strictly delineated into the Incident Commander (IC), who orchestrates the overall response; the Communications Lead, who manages stakeholder updates; and the Operations Lead, who focuses on immediate mitigation[2].

Despite these structured human frameworks, the actual technical investigation remains profoundly inefficient. A typical outage forces an on-call engineer into a reactive investigative loop characterized by continuous context switching. An alert fires, perhaps indicating a sudden spike in 500 internal server errors, or monitoring platforms reveal that multiple Kubernetes pods have entered a CrashLoopBackOff state[3]. To diagnose this, the engineer must manually traverse multiple disconnected systems. They query telemetry in a monitoring platform like Datadog or Sentry, scan recent code changes in version control repositories like GitHub, check deployment pipelines for configuration drifts, look up service ownership registries, and finally cross-reference project management tools like Jira to find the responsible team[4].

In this scenario, the human engineer is essentially acting as a manual database join operation, attempting to correlate temporal events across disparate platforms to isolate the root cause. This manual traversal of operational data is slow, prone to cognitive biases, and heavily reliant on the undocumented tribal knowledge of senior engineers. The primary bottleneck during the mitigation phase is the time required to accurately identify the blast radius—the total scope of the impacted systems and customers—and the true root cause, the underlying systemic trigger that permitted the failure to occur. The proposed integration of connected data through an autonomous, graph-based agent aims to eliminate this manual correlation, transforming the engineer's role from data gatherer to strategic decision-maker.

The Evolution and Limitations of Traditional Root Cause Analysis

Root Cause Analysis (RCA) is the structured, retrospective investigation conducted to uncover the underlying trigger of an incident, rather than merely treating its surface symptoms[1]. In the SRE domain, human error is virtually never accepted as a root cause; instead, it is viewed as a symptom exposing gaps in tooling, testing processes, or system design[1]. The objective of an RCA is systemic improvement and prevention, not the assignment of blame, which forms the cornerstone of the "blameless postmortem" culture[1].

Historically, RCA relied on linear deductive methodologies. The most prevalent technique, the "5 Whys," involves asking "why" repeatedly to drill down through layers of symptoms until an actionable root cause is identified[1]. While this method is highly successful for linear, single-cause problems, achieving an 89% success rate in such scenarios, its efficacy drops to a mere 34% when dealing with multi-cause issues[5]. Cloud-native architectures rarely experience simple, linear failures.

In distributed microservice environments, a single user request might traverse dozens of interdependent microservices across multiple availability zones[4]. The infrastructure is highly ephemeral; crashed containers are replaced automatically by orchestration engines, often destroying localized logs and evidence before they can be analyzed[4]. Furthermore, intricate dependency chains create vast blast radii, allowing a single misconfigured environment variable to cascade into widespread, seemingly unrelated system failures[3].

To address these complex failures, organizations employ a variety of traditional RCA methodologies, though each demonstrates significant limitations when applied to highly dynamic, cloud-native environments.

RCA Methodology	Operational Mechanism	Strengths	Cloud-Native Limitations
5 Whys	Linear causal chain tracing from symptom to trigger by repeatedly asking "why"[1].	Simple, rapid execution for isolated, code-push failures or config errors[3].	Fails to capture converging failure modes, parallel contributing factors, or systemic cascading errors[3].
Fishbone (Ishikawa) Diagram	Categorizes causes into broad domains such as People, Process, Technology, Environment, and	Excellent for multi-team brainstorming, structured broad coverage, and complex	Highly manual and collaborative; typically performed post-incident rather than dynamically during active

	Measurement[1].	multi-cause incidents[3].	mitigation[1].
Fault Tree Analysis (FTA)	Top-down, deductive mapping of logical relationships using boolean AND/OR gates[3].	Maps complex incidents where multiple interdependent conditions must be true simultaneously to trigger failure[4].	Assumes a static architecture; struggles to maintain relevance in ephemeral, continuously deploying environments[4].
Pareto Analysis	Statistical technique prioritizing the most frequent failure modes (the 80/20 rule)[3].	Highly effective for portfolio prioritization, backlog triage, and identifying recurring patterns[3].	Offers no diagnostic utility for single, isolated incident investigation during an active outage[3].
Timeline Analysis	Reconstructs the exact chronological sequence of events leading to the incident[4].	Correlates time across distributed systems, which is essential to reveal causality in microservices[4].	Highly susceptible to monitoring noise, normal system fluctuations, and clock skew across distributed nodes[4].

The fundamental limitation of these traditional RCA methodologies is that they assume the ability to inspect a static, failed system after the fact. In cloud environments, auto-scaling events constantly alter the infrastructure during the incident, cloud provider APIs impose rate limits that slow investigation, and cross-account incidents require multiple sets of credentials just to gather telemetry[4]. Consequently, modern reliability engineering has shifted toward continuous, automated causal inference using dynamic dependency topologies.

Academic Frameworks for Microservice Causal Discovery

The premise of utilizing structural dependencies and advanced machine learning to automate root cause analysis has driven extensive academic research. These frameworks attempt to mathematically model the propagation of faults without requiring predefined, human-authored domain rules, relying instead on telemetry data to construct causal graphs.

Academic research heavily utilizes causal discovery algorithms to build these graphs from time-series metrics. Algorithms such as the PC-algorithm, Greedy Equivalence Search (GES), Fast GES (fGES), and DirectLiNGAM are frequently deployed to estimate causal structures[7].

Once the graph is constructed, scoring methods are applied to traverse the graph and locate the most probable root cause node.
The evaluation of the current state-of-the-art reveals a diverse array of methodologies aimed at solving the microservice RCA challenge.

Framework	Methodological Approach	Mechanism for Root Cause Localization
MicroDiag	Builds a causal graph using the DirectLiNGAM algorithm based on observational telemetry[7].	Applies PageRank algorithms to determine the location of the root cause within the causal graph[8].
MicroCause	Utilizes the PCMCI algorithm to construct the causal dependency graph from raw time-series metrics[7].	Employs a temporal cause-oriented random walk, assuming the most frequently visited node is the root cause[7].
RUN	Leverages Neural Granger Causal Discovery (NGCD) paired with contrastive learning to build the causal graph[7].	Recommends the top-k root causes using a PageRank algorithm customized with a personalized vector[8].
CHASE	Models multimodal data using a Causal Heterogeneous Graph-Based framework (hypergraphs)[10].	Fuses instance-level anomaly detection with heterogeneous message passing across trace, metric, and log data[10].
CIRCA	Abandons automated discovery, constructing the causal graph from call graphs and operator domain knowledge[7].	Formulates failure as an intervention and performs regression-based hypothesis testing on metric distributions[7].
BARO	Bypasses causal graph construction entirely, analyzing pre-failure and post-failure data distributions[8].	Uses a hypothesis testing technique based on median and interquartile range (IQR) to identify anomalous nodes[8].

Despite mathematical rigor, these academic models struggle when deployed in dynamic, real-world production environments. The primary challenges outlined in recent literature include cascading fault propagation, normal fluctuation confusion, and dataset deficiencies.

In a real microservice ecosystem, faults propagate rapidly through inter-service dependencies. A fault in a downstream database can cascade upward, masking the original trigger behind secondary anomalies in the API gateway layer[6]. Existing models, particularly those based on Graph Attention Networks (GATs), often focus too heavily on aggregating these fault symptoms rather than accurately modeling the temporal causal propagation pathways[6].

Furthermore, dynamic fluctuations in normal microservice operations severely hinder algorithmic accuracy. Daily peak-hour traffic spikes, stochastic inter-service interactions, and general monitoring noise introduce metric deviations that heavily resemble fault signals[6]. As the system continuously evolves through architectural adjustments and deployments, it triggers "concept drift," where the statistical distribution of normal behavior shifts over time, diverging from the model's learned representations[6]. Many multi-modal RCA methods fail to distinguish these normal fluctuations from genuine anomalies, leading to unstable and unreliable diagnoses[6].

Finally, academic methods are frequently evaluated on synthetic datasets that fail to replicate the incomplete, noisy reality of enterprise systems, meaning their reported efficacy rarely translates to production value[6].

The Commercial Landscape for Automated RCA

While academic models focus heavily on discovering causality purely from metrics, the commercial landscape has favored deterministic topology mapping combined with full-stack instrumentation. The most mature commercial implementation of graph-based RCA is found in the Dynatrace observability ecosystem, which operates on a unified strategy anchored by OneAgent, Smartscape, and Davis AI.

Dynatrace circumvents the need to guess causal relationships from telemetry by utilizing OneAgent to auto-instrument hosts, containers, processes, and services[13]. This instrumentation feeds a unified, real-time dependency graph known as Smartscape, which maps dependencies vertically from the underlying infrastructure up to the application layer, and horizontally across service-to-service communications[13]. Smartscape serves as the deterministic foundation for Dynatrace's hypermodal engine, Davis AI.

Rather than relying purely on correlation or semantic similarity, Davis AI leverages the Smartscape topology to conduct continuous fault tree analysis[13]. When infrastructure anomalies occur, Davis evaluates millions of dependencies to arrive at the most probable root cause. A critical feature of this system is its approach to event processing and analysis, which prevents the alert fatigue that typically cripples SRE teams during a major outage.

Dynatrace processes events through multi-step normalization, aggregation, and deduplication workflows[16]. Deduplication over time groups fluctuating violation events into a single problem entity. More importantly, deduplication over the causal topology allows Dynatrace to recognize when multiple alerts are symptoms of the same underlying issue. For instance, if an infrastructure problem causes a performance degradation that subsequently triggers alerts across four dependent microservices, Davis AI analyzes the causal topology, concludes they

belong to the same anomaly, and groups all events into a single, actionable problem ticket[14].
To manage long-lasting incidents, Dynatrace enforces a strict problem lifecycle. A problem lifespan is defined by the individual Davis events it contains, and if a problem remains open for longer than 90 minutes, no new events are merged into it to prevent the system from collecting unrelated information for perpetual issues[14]. Furthermore, Dynatrace allows engineers to control event ingestion using control properties, such as dt.event.allow_davis_merge to dictate whether an event can be combined into a larger problem, or dt.event.suppress_problem to store an event without triggering correlation and alerting[16].
Datadog presents another commercial approach through its BitsAI assistant. When an alert fires, BitsAI autonomously launches an investigation, generating multiple root cause hypotheses[5]. It tests these hypotheses by querying data, analyzing logs and traces, and checking recent Kubernetes events, classifying each lead as confirmed, rejected, or unverified[5]. BitsAI draws on episodic memory from past alerts to recognize patterns, allowing continuous learning as human operators validate or correct its conclusions[5].
Despite the advanced capabilities of these platforms, a significant functional gap remains in the commercial sector: the boundary of brokered execution. Platforms like Dynatrace and Datadog are highly effective systems of insight; they excel at identifying the problem and telling operators what is wrong[13]. However, the actual remediation—restarting a workload, rolling back a deployment, or applying a Terraform fix—must still be executed by a human engineer with standing access[5]. Dynatrace does not broker per-task production credentials, does not gate execution behind per-environment approval modes, and does not natively sandbox execution scripts[13]. To bridge this gap, organizations must integrate external brokered execution layers, such as CloudThinker, which consume Dynatrace's causal insights and broker the necessary scoped credentials to execute safe, audited changes[13].
Moreover, these observability platforms focus almost exclusively on operational telemetry. They often lack deep integration with the organization's business logic layer, rendering them incapable of seamlessly connecting a runtime memory leak to the specific human developer who approved the code, or mapping an infrastructure node directly to a specific enterprise customer contract to determine business impact.

The Connected Data Paradigm: Moving Beyond Vector Search

The inability to connect operational failures to human context and business logic highlights the limitations of standard data retrieval systems in incident management. The prevailing trend in artificial intelligence development relies heavily on Retrieval-Augmented Generation (RAG) powered by vector databases. Vector search excels at semantic similarity—finding text that conceptually matches a query—but it fundamentally cannot compute structural dependencies[17]. If an SRE asks a vector database, "Which customer is impacted if the authentication microservice fails?" the system will retrieve documents where the words "authentication" and "customer" appear near each other. It cannot definitively trace the exact runtime architecture to answer the question, because that relationship relies on deterministic structural links, not

semantic proximity. Vector search cannot connect an error log to an on-call engineer through five intermediate layers of infrastructure, version control, and project management[17].
To solve this, enterprise architecture must embrace property graphs and temporal knowledge graphs. By structuring the enterprise ecosystem within a high-performance graph database like FalkorDB, the system preserves explicit formatting, types, and hierarchical relationships[19]. This architecture treats the entire repository, telemetry stream, and personnel directory as a unified, queryable entity. This methodology, often termed GraphRAG, replaces or augments the vector retrieval step with graph-based structural retrieval[17].
Graph databases are exact. The shift from retrieving similar text to traversing structured knowledge changes the class of problems an AI agent can solve[17]. This creates two complementary indices: a structural index (the graph) that knows dependencies, call hierarchies, and ownership boundaries, and a semantic index (the vectors) that understands intent and conceptual proximity[19]. When an alert fires, a graph traversal provides a definitive provenance chain, ensuring the resulting analysis is grounded in verifiable architectural facts rather than LLM hallucinations[17].

Requirement Document: The Autonomous SRE Sentinel System

To mature the conceptual "Autonomous Outage Blast Radius SRE Agent" into a grounded, production-grade system, its architecture must be strictly defined. The following requirements document outlines the functional and structural specifications for a graph-backed incident response system (the SRE Sentinel) designed to minimize Mean Time to Resolution (MTTR) by acting on connected data.

1. Ontological Model and Graph Architecture

The SRE Sentinel relies on a strictly typed temporal knowledge graph stored in a high-performance, memory-optimized graph database (e.g., FalkorDB)[17]. The graph is populated through automated knowledge extraction pipelines (e.g., Graphiti) that parse application specifications, infrastructure configurations, and organizational directories into nodes and typed edges[17].
The system ontology mandates the following deterministic node and edge relationships:

Source Node	Directed Edge	Target Node	Operational Context
Alert	[:FIRED_ON]	Microservice	Maps the monitoring signal to the logical application tier, initiating the traversal.

Microservice	[:RUNS_COMMIT]	Commit	Links the running infrastructure state to the exact Git hash in the version control system.
PullRequest	[:CONTAINS]	Commit	Groups individual code changes into collaborative review units, indicating intent.
PullRequest	[:IMPLEMENTS]	Task	Connects code changes to business logic tracking, mapping Git history to Jira or Asana tickets.
Employee	[:ASSIGNED_TO]	Task	Identifies the author or owner of the business logic, moving from systems to personnel.
Employee	[:REVIEWED]	PullRequest	Identifies the secondary approver responsible for quality control and peer review.
Microservice	[:SERVES]	EnterpriseCustomer	Maps infrastructure directly to revenue-generating contracts to quantify blast radius and SLA risk.

2. The Functional Execution Loop

The SRE Sentinel operates on an automated, multi-stage execution loop triggered by the receipt of an anomaly signal. The objective is to move from signal ingestion to a verified remediation package in sub-seconds.

2.1 Ingestion and Triage (System-1 Routing)

The system must expose a highly available webhook endpoint to ingest real-time payloads from monitoring systems (e.g., a mock Sentry or Datadog alert). Upon receipt, a lightweight, high-speed classifier (a System-1 decision model) parses the error signature to identify the target microservice node[18]. This rapid routing phase normalizes incoming data and prevents deep graph traversals for known, noisy metrics or low-priority anomalies.

2.2 Deep Graph Traversal and Root Cause Identification

Once the initial anomaly node is located, the system executes a standardized graph query (e.g., a Cypher query) via a Model Context Protocol (MCP) server integration[18]. The traversal must efficiently execute a minimum 6-hop path to connect the operational failure to the human and business context. The database engine traverses the graph in reverse from the point of failure to identify upstream dependencies, isolating the specific Commit node that introduced the instability.

2.3 Blast Radius and Impact Calculation

Simultaneously, the query traverses downstream to calculate the blast radius. The system queries the [:SERVES] edges associated with the affected microservice and all dependent services. By aggregating the EnterpriseCustomer nodes linked to the failing paths, the system quantifies the exact business impact, prioritizing mitigation efforts based on high-value contracts and SLA thresholds.

2.4 Remediation Packaging and Brokered Action

The SRE Sentinel does not execute commands unilaterally. Instead, it generates a verified remediation package that includes the precise Git command required to safely revert the breaking commit (e.g., git revert <SHA>), an impact brief summarizing the blast radius, and targeted communication templates. The agent identifies the on-call engineer and the original author of the breaking code, dispatching a direct notification.

Execution is handled through a brokered access model. The system provisions scoped, temporary credentials strictly limited to the necessary rollback task, validating the execution in a sandbox environment and recording a tamper-evident audit log[13].

3. Engineering Realities and Technical Implementation

Transitioning this conceptual architecture into a reliable enterprise tool requires addressing significant engineering challenges surrounding graph maintenance, algorithmic processing, and system security.

3.1 Managing Graph Freshness and Scalability

An outdated dependency graph provides agents with false confidence, resulting in hallucinated remediation steps[19]. Microservice architectures change continuously. Therefore, the indexing layer must support incremental updates. Re-indexing the entire enterprise corpus on every code

commit is computationally prohibitive[17]. A database like FalkorDB natively handles incremental indexing through sparse matrix representations[17]. When an edge mutates (e.g., a new deployment), the system captures the mutation, expands to the affected neighborhood, and recomputes the impacted subgraph without exporting unrelated components[22].

Performance constraints dictate that the graph database must handle single-edge writes in sub-millisecond times[17]. Multi-hop traversal depth multiplies result sets exponentially, requiring memory-optimized architectures to prevent query timeouts within orchestration tools like N8N[17].

3.2 Topological Sorting, Cycle Detection, and Graph Imperfections

Real production graphs are rarely perfect Directed Acyclic Graphs (DAGs). They are often incomplete, disconnected, and contain cyclic dependencies where microservices rely on each other[22]. The SRE Sentinel must push dependency analysis as deeply into the database engine as possible[22].

When generating a rollback plan, the system must translate dependency constraints into an execution plan using topological sort algorithms. Native database algorithms must evaluate the dependency order. If a proposed rollback introduces a dependency cycle, the algorithm must fail early and flag the remediation package as unsafe[22]. Furthermore, if the graph incorporates relationships probabilistically inferred by Large Language Models, these low-confidence edges may create implied cycles that must be isolated during topological sorting to ensure deterministic execution[22].

3.3 Security, Governance, and Blast Radius Protection

A comprehensive context engine that maps code dependencies, vulnerabilities, and customer connections serves as a highly detailed penetration testing roadmap if compromised[19]. Security must be a primary architectural concern.

Agents querying the context engine must strictly adhere to the same repository-level permissions granted to the requesting engineer[19]. The system must prevent context leakage across tenant or team boundaries through strict graph query constraints. Furthermore, building Cypher queries through simple string concatenation is a severe security risk that invites query injection attacks[18]. The system must mandate the use of parameterized syntax or code-level filtering to ensure that malicious alert payloads cannot manipulate the database traversal[18].

Creating the Learning Organization: Episodic Memory in RCA

In mature SRE organizations, an incident is not considered resolved when the dashboard turns green; it is resolved when the system has been fortified against recurrence. This relies on the "Learning Team" concept driven by blameless postmortems[3]. If the same class of incident recurs despite completed postmortems, it indicates a failure in documentation and follow-through[3].

The implementation of a temporal knowledge graph fundamentally enhances organizational learning. A graph database enables the creation of an AI agent memory system categorized into episodic and semantic memory[17]. Semantic memory stores domain facts as persistent graph structures, while episodic memory records past interactions and incidents as timestamped event

nodes linked to the core infrastructure entities[17].
When the SRE Sentinel conducts an investigation, the entire context—the alerts, the cascading failure paths, the identified root cause, and the implemented remediation—is written back to the graph database[17]. When future incidents occur, the agent natively queries its own history, surfacing past failures that share structural similarities. If a database cluster experiences latency matching a failure pattern from six months prior, the agent retrieves the historical postmortem context, instantly bridging the gap between past documentation and present mitigation.

Conclusion

The proposition of an Autonomous "Outage Blast Radius" SRE Agent targets the most persistent inefficiencies in modern incident response. Current operational models trap engineers in manual, reactive loops, forcing them to correlate telemetry, code repositories, and personnel registries under extreme duress. While academic models push the boundaries of causal discovery from time-series metrics, they remain vulnerable to the dynamic fluctuations and cascading failures inherent in microservice architectures. Conversely, commercial observability platforms provide exceptional topological insight but stop short of brokering cross-domain remediations tied to business logic and human workflows.
By anchoring incident management within a high-performance graph database like FalkorDB, the SRE Sentinel bridges these gaps. It transforms root cause analysis from a chaotic investigative effort into a precise, deterministic mathematical traversal. The architecture maps the explicit relationships between infrastructure alerts, code commits, project tasks, and enterprise customers, enabling sub-second blast radius calculations. To realize this system in production, engineering teams must implement robust incremental indexing to guarantee graph freshness, push topological sorting into the database engine for cycle detection, and strictly enforce brokered, sandboxed execution for automated remediations. When deployed with rigorous security and temporal memory capabilities, this connected data approach ensures that organizations do not merely survive incidents, but systematically learn from them, fundamentally reducing operational toil and accelerating resolution.

Works cited

1. Root Cause Analysis - KodeKloud, https://notes.kodekloud.com/docs/Fundamentals-of-SRE/Incident-Management/Root-Cause-Analysis/page
2. Root Cause Analysis for Probing Incident - Google SRE, https://sre.google/workbook/incident-response/
3. 6 Root Cause Analysis Examples for SRE Teams - Coralogix, https://coralogix.com/guides/root-cause-analysis-examples/
4. Root Cause Analysis for SREs: 5 Whys to AI (2026) - Arvo AI - Aurora, https://www.aurorasre.ai/blog/root-cause-analysis-complete-guide-sres
5. Root Cause Analysis: The “Learning” Layer of SRE | by Samuel A Pio, https://medium.com/@samuel.a.pio/root-cause-analysis-the-learning-layer-of-sre-354e8fe96db9
6. DynaCausal: Dynamic Causality-Aware Root Cause Analysis ... - arXiv,

https://arxiv.org/html/2510.22613v1
7. Root Cause Analysis for Microservices based on Causal Inference, https://arxiv.org/pdf/2408.13729
8. Root Cause Analysis for Microservices based on Causal Inference, https://arxiv.org/html/2408.13729v1
9. Root Cause Analysis In Microservice Using Neural Granger Causal, https://arxiv.org/abs/2402.01140
10. CHASE: A Causal Heterogeneous Graph based Framework for Root, https://arxiv.org/html/2406.19711v1
11. Anomaly Detection and Root Cause Analysis for Microservice Systems, https://arxiv.org/abs/2606.09942
12. Root Cause Analysis for Microservice System based on Causal, https://arxiv.org/abs/2408.13729
13. CloudThinker vs Dynatrace: AIOps Insight vs AgenticOps Execution, https://www.cloudthinker.io/vs/dynatrace
14. Root cause analysis concepts — Dynatrace Docs, https://docs.dynatrace.com/docs/dynatrace-intelligence/root-cause-analysis/concepts
15. Smartscape monitoring & observability | Dynatrace Hub, https://www.dynatrace.com/hub/detail/smartscape-1/
16. Event analysis and correlation - Dynatrace Documentation, https://docs.dynatrace.com/docs/dynatrace-intelligence/root-cause-analysis/event-analysis-and-correlation
17. Graph Database AI Agents: GraphRAG & Memory Guide - FalkorDB, https://www.falkordb.com/blog/graph-database-ai-agents/
18. How to Build an AI RAG Workflow using FalkorDB + N8N + Graphiti, https://www.falkordb.com/blog/ai-rag-workflow-falkordb-n8n-graphiti/
19. Source Context Management: Turn Repos Into Query Engines, https://www.harness.io/blog/your-repo-is-a-knowledge-graph-you-just-dont-query-it-yet
20. Graph-Based Agent Memory: A Complete Guide to Structure, https://shibuiyusuke.medium.com/graph-based-agent-memory-a-complete-guide-to-structure-retrieval-and-evolution-6f91637ad078
21. Platano78/faulkner-db: Temporal Knowledge Graph System for, https://github.com/Platano78/faulkner-db
22. Topological Sort Algorithm: A Practical Guide for 2026 - FalkorDB, https://www.falkordb.com/blog/topological-sort-algorithm-2/